SITUAÇÃO APRENDIZAGEM			
Curso	CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS		
Unidade Curricular:	Versionamento e Colaboração		
SA Meu primeiro Repositório de Assets e Scripts			
Estratégia de Aprendizagem Desafiadora			
( ) Situação-Problema	( ) Estudo de Caso	(X) Projeto	( ) Pesquisa Aplicada

CONTEXTUALIZAÇÃO:

Você foi contratado por um estúdio independente para organizar o **Game Design Document (GDD)** e os primeiros arquivos de um protótipo de jogo. Sua missão é garantir que toda a equipe tenha acesso às versões mais recentes do projeto através do GitHub.

DESAFIO:
Criar e configurar um repositório público no GitHub que contenha a estrutura inicial de um projeto de jogo, aplicando as boas práticas de versionamento.

PASSOS:

1. **Preparação do Ambiente:**
    1. Crie uma pasta no seu computador chamada projeto-jogo-tecnico.
    2. Dentro dela, crie a seguinte estrutura de pastas:
    3. /docs (para o GDD e textos).
    4. /assets (para imagens, sprites ou sons).
    5. /scripts (para códigos em C# ou GDScript).
2. **Versionamento Local (Git):**
    1. Inicie o repositório localmente (git init).
    2. Crie um arquivo chamado README.md na raiz, descrevendo o nome do seu jogo e a equipe.
    3. Crie um arquivo .gitignore configurado para ignorar arquivos temporários (ex: .DS_Store ou pastas de cache).
    4. Realize o primeiro commit: "feat: estrutura inicial do projeto".
3. **Publicação no GitHub:**
    1. Crie um repositório remoto com o nome meu-primeiro-jogo-git.
    2. Conecte seu repositório local ao remoto e faça o push dos arquivos.
4. **Simulação de Atualização:**
    1. Adicione uma imagem qualquer na pasta /assets e um arquivo de texto com uma ideia de mecânica na pasta /docs.
    2. Realize um novo commit com uma mensagem clara: "docs: adiciona rascunho da mecanica de pulo".

3. Envie as alterações para o GitHub.

5. **Criar uma Branch de Recurso:**
    1. No terminal, crie e mude para uma nova branch chamada feature/fase-deserto.
    2. Comando: git checkout -b feature/fase-deserto

6. **Desenvolver a Novidade:**
    1. Dentro da pasta /assets, crie um arquivo de texto chamado concept-deserto.txt.
    2. Escreva brevemente como será o cenário (ex: "Clima árido, cactos que dão dano").
    3. Adicione e salve essa alteração apenas nesta branch: git add . e git commit -m "feat: rascunho visual da fase do deserto".

7. **A "Viagem no Tempo":**
    1. Volte para a branch principal: git checkout main.
    2. Observe que o arquivo concept-deserto.txt **sumiu** da sua pasta. Não entre em pânico! Ele está seguro na outra branch.
    3. Volte para a branch do deserto: git checkout feature/fase-deserto e veja o arquivo reaparecer.

8. **Fusão (Merge):**
    1. Após aprovação da equipe, una a fase do deserto à versão principal.
    2. Primeiro, volte para a main: git checkout main.
    3. Depois, traga as mudanças: git merge feature/fase-deserto.
    4. Envie tudo para o GitHub: git push origin main.

RESULTADOS ESPERADOS:

Repositório criado
Arquivos compartilhados

CRITÉRIOS DE AVALIAÇÃO:

- O repositório está público e acessível?
- As mensagens de commit são claras e padronizadas?
- O arquivo .gitignore foi incluído corretamente?
- O aluno consegue transitar entre as branches sem se perder?
- A estrutura de pastas está organizada?
- O histórico do GitHub mostra a mesclagem (merge) concluída?

Guia de Comandos Git: Sobrevivência no Desenvolvimento

Ação	Comando	O que faz?
Iniciar	git init	Cria o "rastreador" do Git na pasta atual.
Status	git status	Lista o que foi alterado ou ainda não foi salvo.
Preparar	git add .	"Coloca na caixa" todos os arquivos novos/alterados.
Salvar	git commit -m "msg"	"Fecha a caixa" e cria uma versão oficial no histórico.
Conectar	git remote add origin URL	Diz ao Git local onde o GitHub (servidor) está.
Enviar	git push -u origin main	Sobe suas versões salvas para a nuvem pela 1ª vez.
Atualizar	git pull	Baixa as novidades que seus colegas enviaram.

(Branches)

Ação	Comando	O que faz?
Criar Ramo	git branch nome-da-fase	Cria uma ramificação para testar novas ideias.
Trocar	git checkout nome-da-fase	Pula para a branch escolhida (muda os arquivos da pasta).
Criar e Ir	git checkout -b nome	Faz as duas ações acima de uma só vez.
Listar	git branch	Mostra em qual branch você está agora.

Unir	git merge nome-da-fase	Traz as mudanças da branch para a main (faça isso na main!).

Modelo simplificado de **.gitignore** focado em **Unity**:
gitignore
Arquivos que o Git deve IGNORAR (Não enviar para o GitHub)
.gitignore deve ser criado **antes** do primeiro git add ., para evitar que o Git tente subir arquivos desnecessários.

```
# Pastas de biblioteca e temporárias da Unity
    [Ll]ibrary/
    [Tt]emp/
    [Oo]bj/
    [Bb]uild/
    [Bb]uilds/
    [Logs]/

  # Arquivos de solução do Visual Studio (gerados automaticamente)
    *.csproj
*.sln
*.userprefs

# Arquivos do sistema operacional
    .DS_Store
    Thumbs.db
```

ENTREGA: 19/05/2026